Министерство образования Республики Беларусь
Учреждение образования
«Белорусский государственный университет информатики
и радиоэлектроники»

Специальность «Программная инженерия»

Кафедра инженерной психологии и эргономики

Учебная дисциплина «Метрология, стандартизация и сертификация
(в информационных технологиях)»

Отчет

по лабораторной работе №1
«Метрики размера программ»

Подготовил: Студенты гр. 410902
Грибач Н. Э.
Андала И. О.

Проверил: Болтак С. В.

Минск 2026

1. Цель работы

Изучить метрики размера программ на примере метрик Холстеда, разработать программу-парсер с графическим интерфейсом, которая по исходному тексту программы на языке Groovy рассчитывает шесть базовых и три расширенные метрики Холстеда, и сравнить результаты её работы с ручным расчётом.

2. Теоретические сведения

Основу метрик Холстеда составляют шесть базовых метрик программы:

- η_1 – словарь операторов, число уникальных операторов программы;
- η_2 – словарь операндов, число уникальных операндов программы;
- N_1 – общее число операторов в программе, $N_1 = \Sigma\, f_{1j}$, $j = 1 \ldots \eta_1$;
- N_2 – общее число операндов в программе, $N_2 = \Sigma\, f_{2i}$, $i = 1 \ldots \eta_2$;
- f_{1j} – число вхождений j-го оператора;
- f_{2i} – число вхождений i-го операнда.

На основе базовых метрик рассчитываются производные (расширенные) метрики:

- словарь программы $\eta = \eta_1 + \eta_2$;
- длина программы $N = N_1 + N_2$;
- объём программы $V = N \cdot \log_2 \eta$, бит.

Правила подсчёта, принятые в работе (интерпретация Холстеда с поправкой на язык Groovy):

- операторами считаются символы операций, символ присваивания, символы-разделители «точка», «точка с запятой» и «запятая», а также пара скобок – круглых, квадратных или фигурных – один оператор;
- несколько служебных слов, входящих в состав одной конструкции, считаются одним оператором (if…else, switch…case…default, try…catch…finally, for…in…do, while…do);
- имя объявляемой или вызываемой подпрограммы считается оператором вида «имя( )», скобки вызова отдельно не учитываются;
- операндами считаются переменные, параметры, поля, числовые и строковые литералы, а также значения true, false, null;
- служебные слова объявлений (def, class, имена типов, модификаторы) и метки не относятся ни к операторам, ни к операндам;
- комментарии из анализа исключаются.

3. Код анализируемой программы

В качестве анализируемой программы разработано консольное приложение на языке Groovy – калькулятор с текстовым меню (арифметические операции, факториал, возведение в степень, история вычислений). Объём программы – 97 строк.

```
def reader = System.in.newReader()
def history = []

def readNumber(reader, title) {
    println title
    try {
        return Double.parseDouble(reader.readLine().trim())
    } catch (NumberFormatException e) {
        println 'Это не число: ' + e.message
        return null
    }
}

def apply(a, b, sign) {
    switch (sign) {
        case '+': return a + b
        case '-': return a - b
        case '*': return a * b
        case '/': return b == 0 ? null : a / b
        case '%': return b == 0 ? null : a % b
        default: return null
    }
}

def factorial(n) {
    def result = 1
    if (n < 2) return result
    for (i in 2..n) {
        result = result * i
    }
    return result
}

def power(base, degree) {
    def value = 1
    def steps = degree < 0 ? -degree : degree
    for (i in 0..<steps) {
        value = value * base
    }
    return degree < 0 ? 1 / value : value
}

def arithmetic(reader, history) {
    def a = readNumber(reader, 'Первое число:')
    def b = readNumber(reader, 'Второе число:')
    println 'Знак операции:'
    def sign = reader.readLine().trim()
    if (a == null || b == null) return 'Операция отменена'
    def value = apply(a, b, sign)
    if (value == null) return 'Некорректная операция'
    def note = a + ' ' + sign + ' ' + b + ' = ' + value
    history << note
    return note
```

```
}

def show(history) {
    if (history.isEmpty()) {
        println 'История пуста'
        return
    }
    history.each { note -> println note }
    println 'Всего операций: ' + history.size()
}

while (true) {
    println '1 - арифметика, 2 - факториал, 3 - степень, 4 - история, 0 - выход'
    def cmd = reader.readLine()
    if (cmd == null) {
        break
    }
    switch (cmd.trim()) {
        case '1':
            println arithmetic(reader, history)
            break
        case '2':
            def n = readNumber(reader, 'Целое число 0..20:')
            def factValue = n == null || n < 0 || n > 20 ? 'Нет данных' : factorial(n
as int)
            history << 'n! = ' + factValue
            println factValue
            break
        case '3':
            def base = readNumber(reader, 'Основание:')
            def degree = readNumber(reader, 'Показатель:')
            def powValue = base == null || degree == null ? 'Нет данных' :
power(base, degree as int)
            history << 'Степень = ' + powValue
            println powValue
            break
        case '4':
            show(history)
            break
        case '0':
            println 'Выход'
            return
        default:
            println 'Нет такой команды'
    }
}
```

4. Результаты ручного расчёта базовых метрик

j	Оператор	f_{1j}	i	Операнд	f_{2i}
1.	:	18	1.	b	12
2.	=	18	2.	reader	12
3.	{ }	16	3.	history	11
4.	return	16	4.	null	11

5.	,	14	5.	a	10
6.	.	13	6.	degree	8
7.	println( )	12	7.	n	8
8.	( )	11	8.	value	8
9.	+	11	9.	0	6
10.	==	9	10.	base	5
11.	?	6	11.	note	5
12.	readNumber( )	6	12.	result	5
13.	break	5	13.	sign	5
14.	if…else	5	14.	1	3
15.	<	4	15.	cmd	3
16.	||	4	16.	factValue	3
17.	*	3	17.	i	3
18.	<<	3	18.	powValue	3
19.	readLine( )	3	19.	' '	2
20.	trim( )	3	20.	'Нет данных'	2
21.	-	2	21.	2	2
22.	/	2	22.	e	2
23.	apply( )	2	23.	steps	2
24.	arithmetic( )	2	24.	title	2
25.	as	2	25.	' = '	1
26.	factorial( )	2	26.	'-'	1
27.	for…in…do	2	27.	'*'	1
28.	power( )	2	28.	'/'	1
29.	show( )	2	29.	'%'	1
30.	switch…case…default	2	30.	'+'	1
31.	->	1	31.	'0'	1

32.	..	1	32.	'1 - арифметика, 2 - факториал, 3 - степень, 4 - история, 0 - выход'	1
33.	..<	1	33.	'1'	1
34.	[ ]	1	34.	'2'	1
35.	%	1	35.	'3'	1
36.	>	1	36.	'4'	1
37.	each( )	1	37.	'Всего операций: '	1
38.	isEmpty( )	1	38.	'Второе число:'	1
39.	newReader( )	1	39.	'Выход'	1
40.	parseDouble( )	1	40.	'Знак операции:'	1
41.	size( )	1	41.	'История пуста'	1
42.	try…catch…finally	1	42.	'Некорректная операция'	1
43.	while…do	1	43.	'Нет такой команды'	1
			44.	'Операция отменена'	1
			45.	'Основание:'	1
			46.	'Первое число:'	1
			47.	'Показатель:'	1
			48.	'Степень = '	1
			49.	'Целое число 0..20:'	1
			50.	'Это не число: '	1
			51.	'n! = '	1
			52.	20	1
			53.	Double	1
			54.	in	1
			55.	message	1
			56.	System	1
			57.	true	1
$\eta_1 = 43$		**$N_1 =$**	**$\eta_2 = 57$**		**$N_2 =$**

	213		**166**

5. Расчёт расширенных метрик

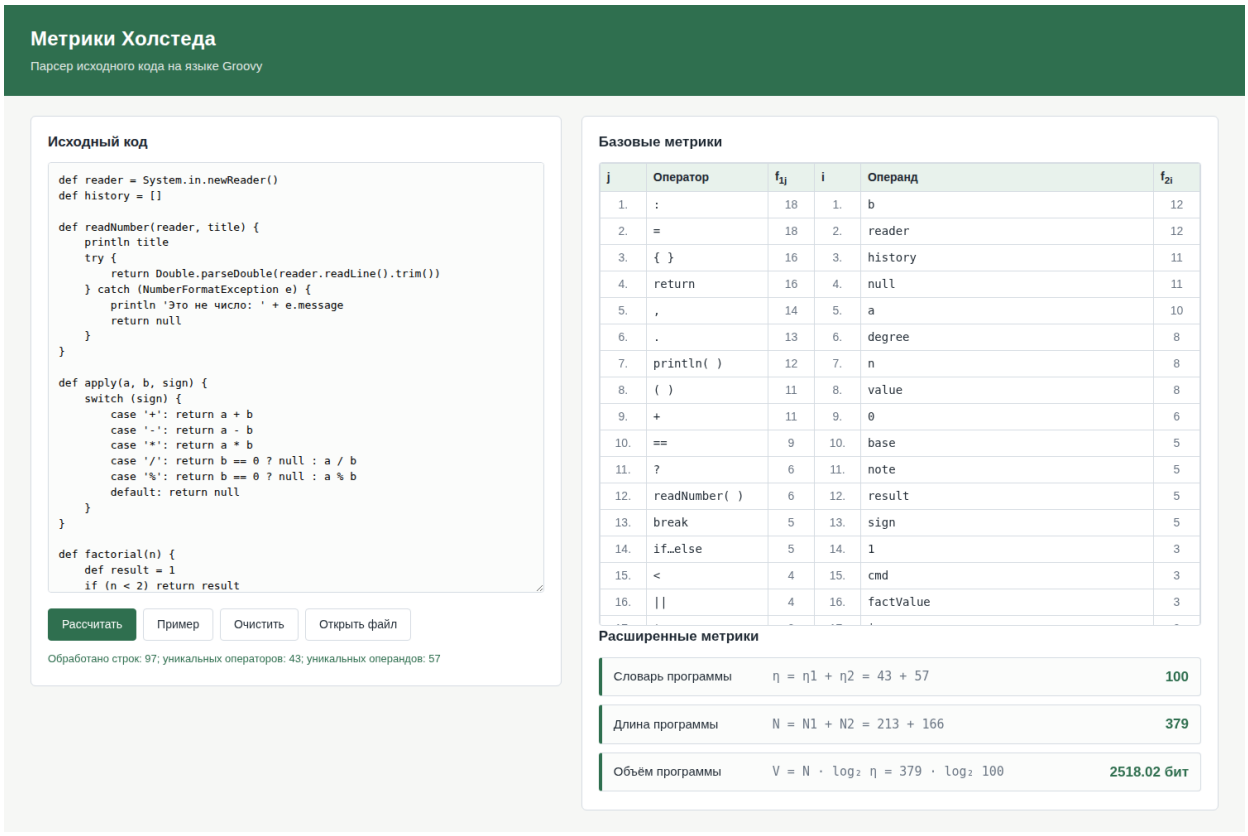
Словарь программы: $\eta = \eta_1 + \eta_2 = 43 + 57 = 100$.

Длина программы: $N = N_1 + N_2 = 213 + 166 = 379$.

Объём программы: $V = N \cdot \log_2 \eta = 379 \cdot \log_2 100 = 379 \cdot 6.6439 \approx 2518.02$ бит.

6. Результат работы программы

Парсер реализован на языке JavaScript, интерфейс – веб-страница. Программа выполняет лексический разбор введённого кода, классифицирует лексемы на операторы и операнды, выводит таблицу базовых метрик и отдельный блок расширенных метрик.



7. Вывод

В ходе работы изучены метрики размера программ на примере метрик Холстеда и разработан парсер исходного кода на языке Groovy, рассчитывающий шесть базовых и три расширенные метрики. Результаты работы программы полностью совпали с ручным расчётом: $\eta_1 = 43$, $\eta_2 = 57$, $N_1 = 213$, $N_2 = 166$, $\eta = 100$, $N = 379$, $V \approx 2518.02$ бит.